

Lab 1

Jooho Oh

01-18-2023

Submission Instructions

This lab (and all labs) will be graded for correctness. To submit the lab:

1. Once all code is completed and saved, in the RStudio ribbon, click Knit > Knit to PDF.
2. If you have trouble knitting **anything** to PDF, please work with the Instructor or GSI to resolve the issue. It is okay to instead knit to HTML, then convert to PDF using Adobe or other software.
3. If there is any handwritten work for the lab, combine the handwritten work with the knitted PDF into a single PDF file.
4. Submit the PDF to Gradescope, **selecting** pages for questions. *You will lose half a point on the lab if you don't select pages.*

Lab 1 is due to Gradescope by Sunday, January 22nd at 11:59PM PT.

Collaboration Policy

Statistics and Data Science is a collaborative activity. Please feel free to work with others on the lab, but you must **write your solutions individually**. If your lab answers are clearly a slightly altered copy of another student's, this is considered cheating.

Simple Linear Regression with SAT Data

In this lab we will work through simple linear regression in R by computing the regression coefficients explicitly from the formulas in Fox 5.1, and by using built in R functions (i.e. `lm`) to accomplish the same task. We will explore the relationship between wealth and SAT score with some simulated data of income, SAT score, and number of books read in the last two years.

Load and check the data

```
sat_data <- read.csv("~/Desktop/spring 2023/stat 151a/lab1/SAT_data.csv")
head(sat_data)
```

```
##   NumBooks   Income   SAT
## 1      52 39.85786  755
## 2      53 34.59518  685
## 3      67 46.81881  896
## 4      67 45.80432  901
## 5      77 58.40879 1093
## 6      70 56.65816 1016
```

Plot Variables of Interest

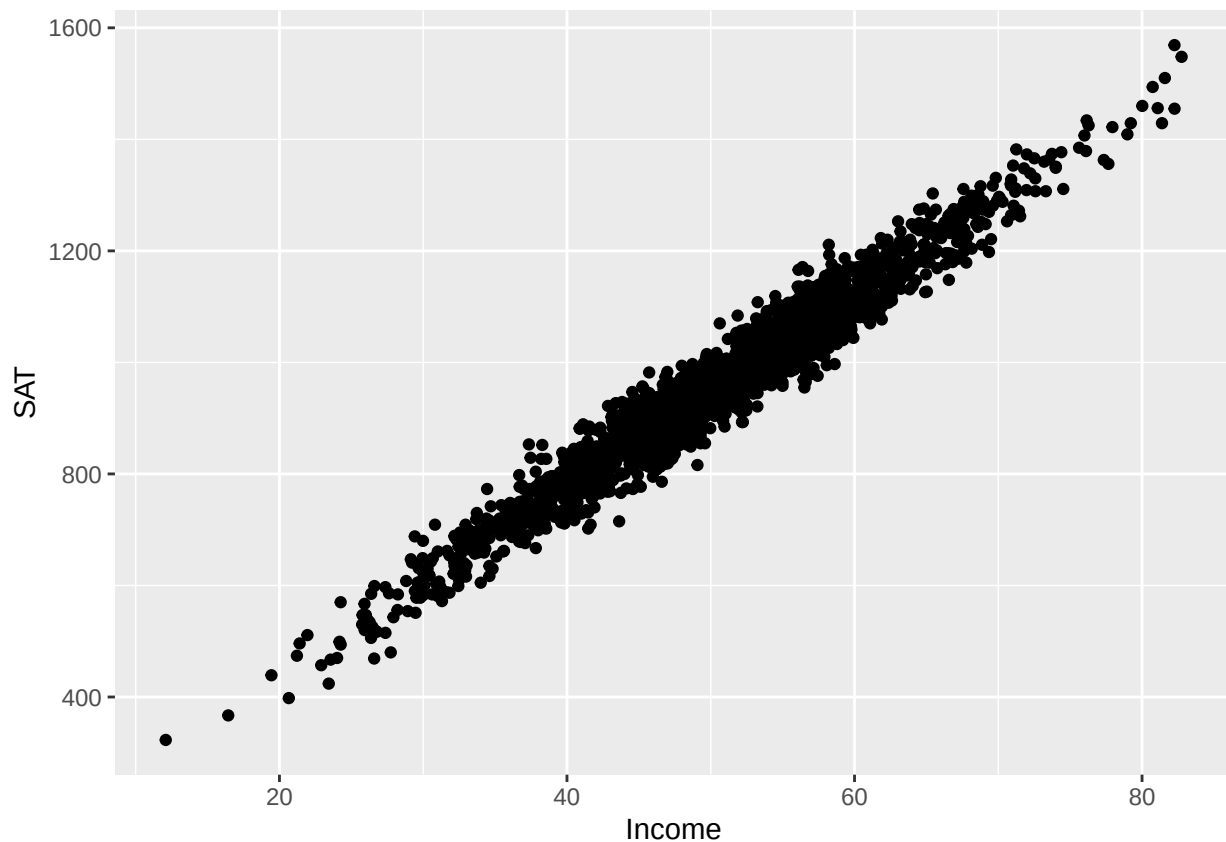
It's always a good idea to visualize your data before doing any modeling. We are interested in the `Income` and `SAT` variables. `SAT` will be the dependent variable of interest.

We can look at the simple bivariate distribution of these columns with a scatterplot.

Q1: Create a scatter plot with the independent variable of interest on the x-axis and the dependent variable of interest on the y-axis. Base R or ggplot2 is fine.

YOUR CODE HERE:

```
ggplot(data = sat_data, aes(x=Income, y=SAT)) + geom_point()
```



Now, we will save the variables to shorthand vectors for later use.

```
y <- sat_data$SAT
x <- sat_data$Income
n <- nrow(sat_data)
```

Simple linear regression from normal equations

Page 85 of Fox walks through how to compute the coefficients for the best fitting line in terms of the least squares criterion.

Solving the normal equations yields the least squares coefficients:

$$a = \bar{Y} - b\bar{X}$$

$$b = \frac{n \sum X_i Y_i - \sum X_i \sum Y_i}{n \sum X_i^2 - (\sum X_i)^2} = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sum (X_i - \bar{X})^2}$$

Thus we see that the slope b can be computed in terms of simple summary statistics, namely the covariance between X and Y and the variance in X , and a follows from the means of Y and X and the value computed for b .

Q2: Please compute b and a according to the formulas above for our data in the following code chunk, storing them as variables named b and a . Make sure to call the objects in the chunk, so that R will print out your answers.

```
### YOUR CODE HERE:
```

```
b <- cov(x,y)/var(x)
b
```

```
## [1] 16.91897
```

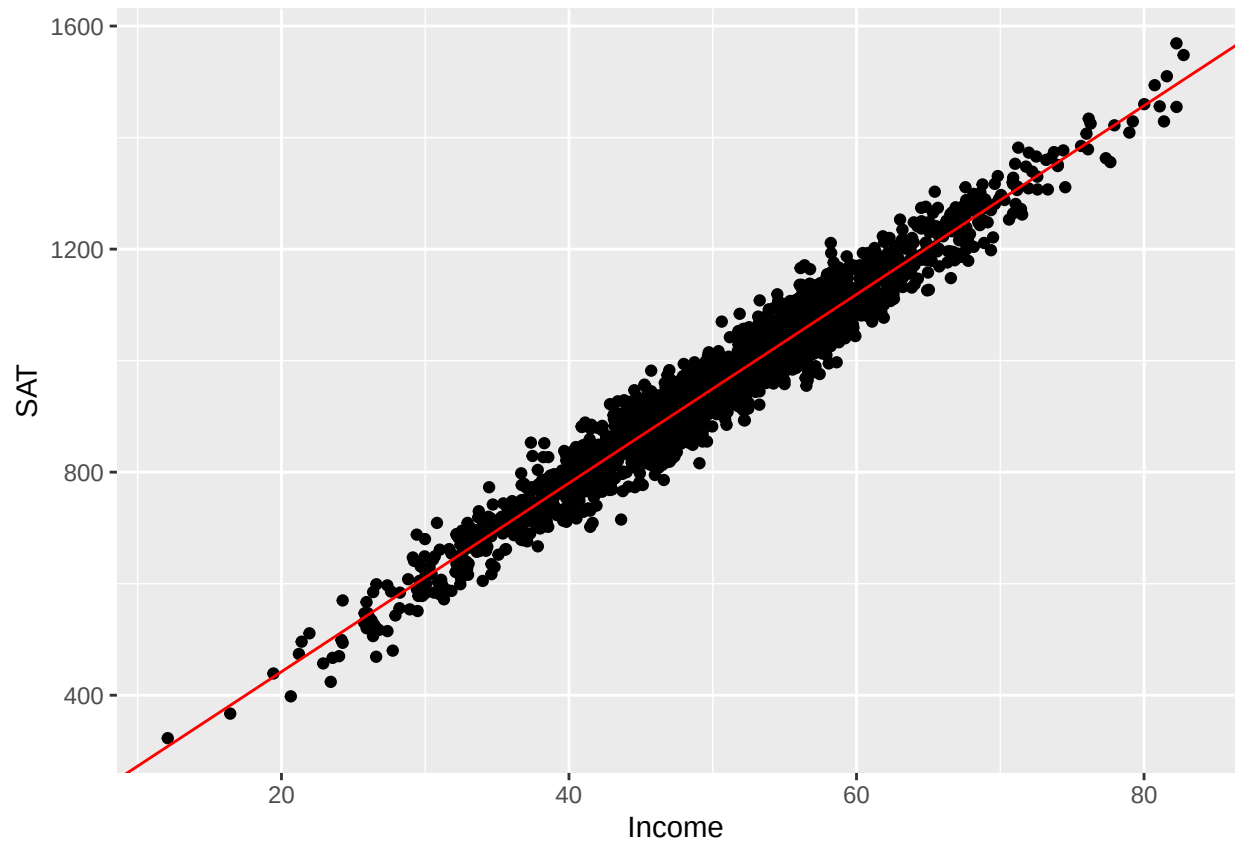
```
a <- mean(y)-b*mean(x)
a
```

```
## [1] 103.7412
```

Q3: Overlay the line of best fit computed in Q2 with the scatterplot from Q1. Make sure to use different colors for scatter points and the line (color or fill aesthetics).

```
### YOUR CODE HERE:
```

```
ggplot(data = sat_data, aes(x=Income, y=SAT)) + geom_point() + geom_abline(slope = b, intercept = a, col
```



Compare to `lm()`

Let's see if we get the exact same results if we had used `lm()` instead of computing the coefficients by hand. Please review the syntax below for using the `lm()` function:

```
model <- lm(SAT ~ Income, data = sat_data)
model

##
## Call:
## lm(formula = SAT ~ Income, data = sat_data)
##
## Coefficients:
## (Intercept)      Income
##      103.74         16.92
```

Residuals

With these coefficients, let's explicitly find the fitted values, residuals, and residual sum of squares (RSS) without using built in functions.

Here is R's built-in way to compute predictions and residuals (two ways):

```
## either works
fitted_r <- fitted(model)
fitted_r <- model$fitted.values

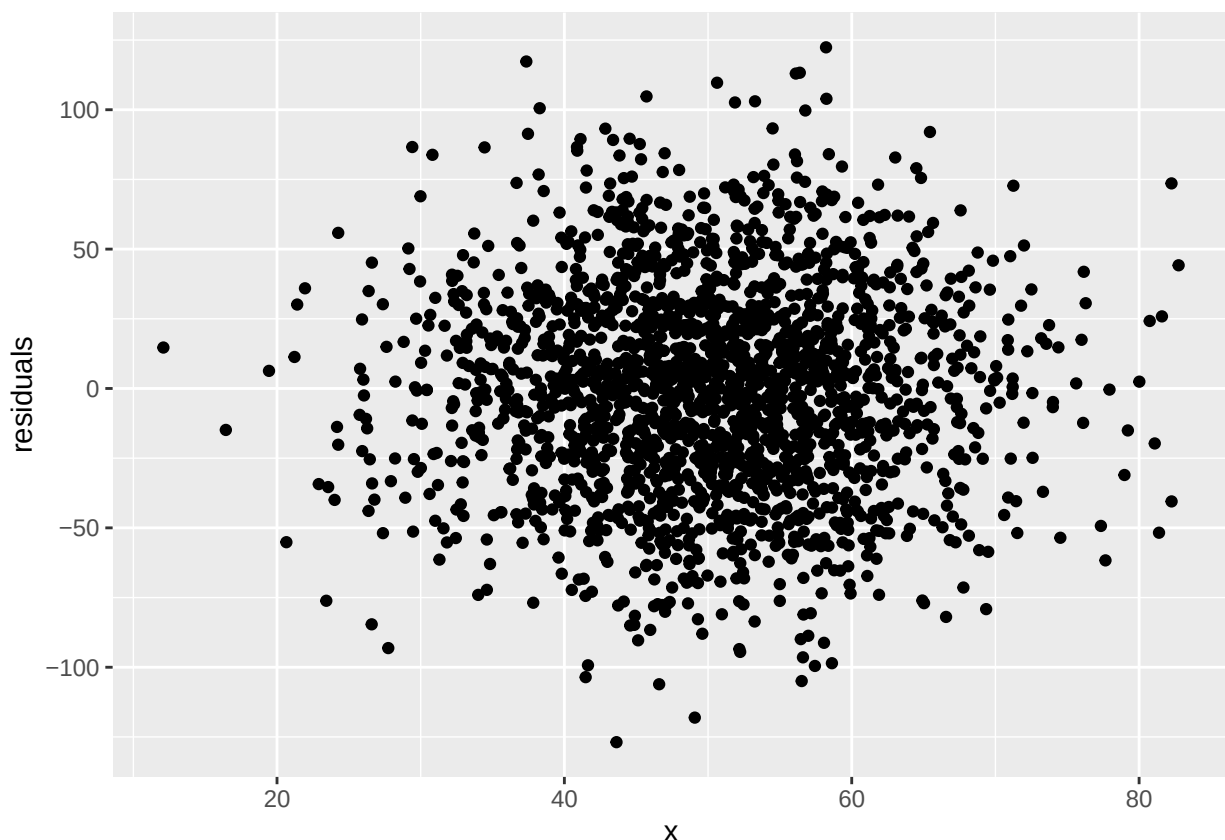
## either works
residuals_r <- residuals(model)
residuals_r <- model$residuals
```

Q4: Manually generate a vector of predicted SAT scores using the computed coefficients, a and b from Q2. Call this vector `fitted_e`. Then, manually create a vector of residuals using `y` and `fitted_e`. Call this vector `residuals_e`.

```
## YOUR CODE HERE:
fitted_e <- a + b*x
residuals_e <- y - fitted_e
```

Q5: Plot the residuals (y-axis) against Income (x-axis). Do you see a relationship between Income and residuals?

```
### YOUR CODE HERE:
ggplot(data.frame(x=x, residuals=residuals_e), aes(x=x, y=residuals)) + geom_point()
```



I think there is no significant relationship between residuals and Income because the shape of the scatter plot is round.

Aside: What is the sum of the residuals?

```
round(sum(residuals_r), 5)
```

```
## [1] 0
```

Answer: 0! (because of the normal equations)

Below we compute the residual sum of squares:

```
### SOLUTION:
```

```
RSS <- sum((residuals_r)^2)
```

```
RSS
```

```
## [1] 2784973
```

Q6: Using RSS object in the prior code chunk, manually compute and report the standard error of the regression (see Fox 87-88).

```
### YOUR ANSWER HERE:
```

```
regression_std_error_e <- sqrt(RSS/(n-2))
```

```
regression_std_error_e
```

```
## [1] 37.33471
```

Note: You may check your answer to Q6 with the built-in R answer:

```
sum_model <- summary(model)
```

```
regression_std_error_r <- sum_model$sigma
```

And recall the following procedure for computing the **correlation coefficient** (R-squared)

```
### Manually:
```

```
TSS <- sum((y - mean(y))^2)
```

```
r_squared_e <- (TSS - RSS) / TSS
```

```
### Via: R
```

```
r_squared_r <- sum_model$r.squared
```

Note that for 1D linear regression, we may also compute r directly using sample moments from the original data:

$$r = \frac{Cov(x, y)}{\sigma_x \sigma_y}$$

where Cov is covariance and σ_x is the standard deviation of $\mathbf{x}$.

Q7: Compute r (or r-squared) by applying the sample moments equation to the data:

```
### YOUR ANSWER HERE:
r <- cov(x,y)/(sd(x)*sd(y))
r
```

```
## [1] 0.9764149
```

Finally, we can easily generate a prediction for a new x value using the `predict()` function.

```
new_income <- data.frame("Income" = 44)
sat_prediction <- predict(model, newdata = new_income)
sat_prediction
```

```
##          1
## 848.1759
```

Note that we will generate a vector of predictions if the data frame is multiple rows.

Q8 (HANDWRITTEN QUESTION):

Consider the least squares fit for **no intercept**:

$$\min_c \sum_i (Y_i - cX_i)^2$$

(ie, we are finding the line of best fit that is constrained to pass through the origin)

The above is minimized when:

$$\hat{c} = \frac{y^\top x}{x^\top x}$$

where x and y are the data vectors. Disprove (by counterexample) that the mean of the residuals is equal to zero.

```
c_hat = sum(y*x)/sum(x*x)
residuals = y-c_hat*x
mean_residual = mean(residuals)
mean_residual
```

```
## [1] 3.953856
```

```
# By using the SAT data, we could get the residual as non-zero value which means it is a counterexample
```

You finished!



Acknowledgements: This document was based on Lab 1 file by Amanda Glazer and Jake Spertus.